

Practice 1: SOFTWARE-DEFINED RADIO PROGRAMMING

Santiago Rodríguez Villabona - 2220387
Juan José De la Rosa Medina - 2202810

https://github.com/santiagoUIS/GNURADIO_COMUIS_2025_2_B1_G7.git

School of Electrical, Electronic, and Telecommunications Engineering
Industrial University of Santander

September 1, 2025

Abstract—Digital communications are central to the development of modern technologies, supporting applications ranging from mobile telephony to real-time data transmission systems. In this context, software-defined radio (SDR) has established itself as a versatile tool for the analysis, simulation, and implementation of communication schemes. The purpose of this practical exercise was to develop skills in programming functions within GNURadio, in order to facilitate interaction with digital signal processing environments. During the lab, the fundamental aspects associated with real-time systems and SDR were identified, including the generation of functions using Python code implementation blocks. Finally, the developed blocks were applied to the processing of real signals, which reinforced the theoretical concepts seen in class and demonstrated the relevance of SDR as an academic and experimental platform.

keywords: *software-defined radio, GNURadio, python block, real-time data, digital signal processing.*

I INTRODUCTION

The evolution of software-defined radio (SDR) has enabled the design and validation of communication systems in the digital domain through the use of flexible programming platforms such as GNURadio. This open-source tool facilitates the construction of flowgraphs that represent signal processing tasks, while also offering the possibility of incorporating custom Python blocks[1].

The main objective of this laboratory was to familiarize students with block programming in GNU Radio, thereby extending the capabilities of the predefined environment. To this end, elementary algorithms were implemented and evaluated, including an accumulator and a differentiator, using graphical visualization tools within the interface to analyze their behavior.

Beyond the implementation of basic algorithms, the use of GNURadio contributed to the consolidation of skills related to modular system design, interaction with predefined and custom blocks, and the validation

of functions in a practical, application-oriented environment. In this way, the practice focused not only on programming, but also on understanding fundamental digital signal processing operations. In addition, Python blocks were developed to compute statistical measures [2] such as the mean, quadratic mean, root mean square (RMS), power spectral density, and standard deviation, thereby strengthening the analysis of signals and noise within the laboratory context.

II METHODOLOGY

This section presents in an orderly and detailed manner the experiments carried out during the laboratory, including simulations and other considerations addressed within the practice.

II-A Implementation of the accumulator block

also known as an integrator block in the digital domain, each sample of its output is the sum of all previous input samples. Mathematically, it can be represented as shown in eq. 1.

$$y[n] = \sum_{k=0}^n x[k] \quad (1)$$

For its implementation, the numpy library is used, which specializes in arithmetic calculations with arrays.

II-B Implementation of the differentiator block

The differentiator block allows discrete-time signals to be analyzed by highlighting rapid changes over time. In this way, the differentiator not only performs a mathematical transformation of the signal, but also provides a practical analysis tool for identifying relevant dynamics in real systems, particularly in applications where high-frequency components carry critical information. Mathematically, it can be expressed as shown in eq. 2.

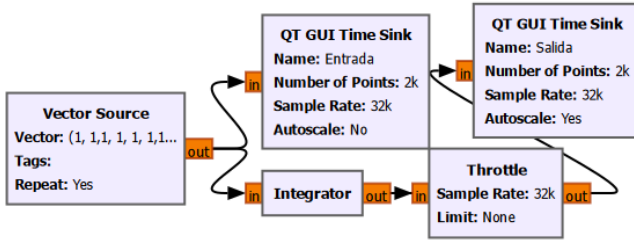


Figure 1. Implementation of the integrator block in GNURadio.

$$y[n] = x[n] - x[n-1] \quad (2)$$

As with the integration block, numpy is used to model the discrete differentiation function.

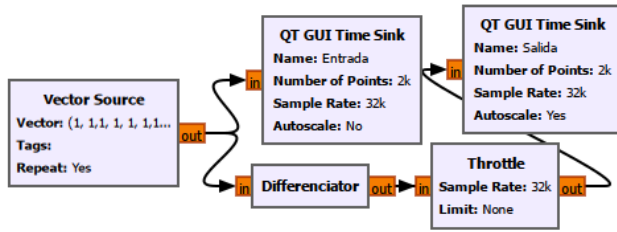


Figure 2. Implementation of the differentiator block in GNURadio.

II-C Implementation of blocks for discrete statistical analysis

In addition to the basic algorithmic blocks, statistical analysis was incorporated through Python blocks to extract key parameters from discrete signals. Metrics such as mean(3), RMS value(4), average power(5) and standard deviation(6) were calculated providing complementary tools to evaluate the behavior of the signals beyond their time-domain representation.

- Mean

$$\bar{x} = \frac{1}{N} \sum_{n=0}^{N-1} x[n] \quad (3)$$

- Root Mean Square (RMS)

$$x_{\text{rms}} = \sqrt{\frac{1}{N} \sum_{n=0}^{N-1} x[n]^2} \quad (4)$$

- Average Power

$$P = \frac{1}{N} \sum_{n=0}^{N-1} |x[n]|^2 \quad (5)$$

- Standard Deviation

$$\sigma_x = \sqrt{\frac{1}{N} \sum_{n=0}^{N-1} (x[n] - \bar{x})^2} \quad (6)$$

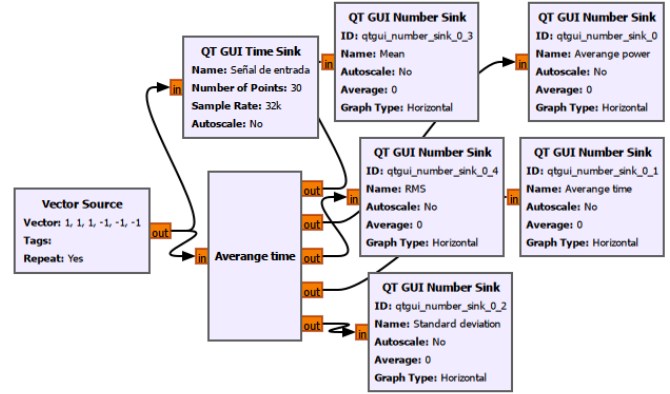


Figure 3. Implementation of the statistical blocks in GNURadio.

II-D White noise addition

To complement this, noise was introduced to study its effect on communication systems, simulating a system closer to reality. Gaussian white noise is chosen due to its statistical properties.

The experiments with the application and statistical blocks are performed again to identify differences due to the presence of noise (these changes can be observed in Fig. 4 5 6).

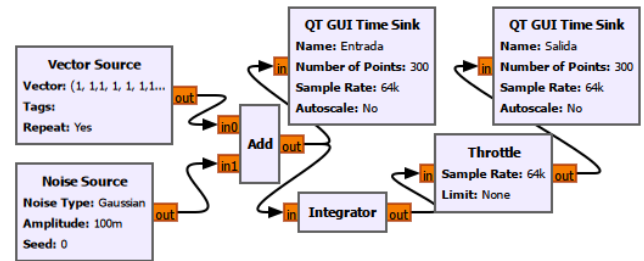


Figure 4. Integrator with input contaminated by white noise.

III EXPERIMENTS AND RESULTS

III-A Application blocks implementation whitout noise

After programming each of the Python blocks, the respective simulations are performed in order to identify the characteristics of a discrete bipolar signal with amplitude $A = 1$. A sampling rate of 32kHz was

simulating a frequency of 0, resulting in an output signal contaminated by small variations, as seen in Fig. 10.

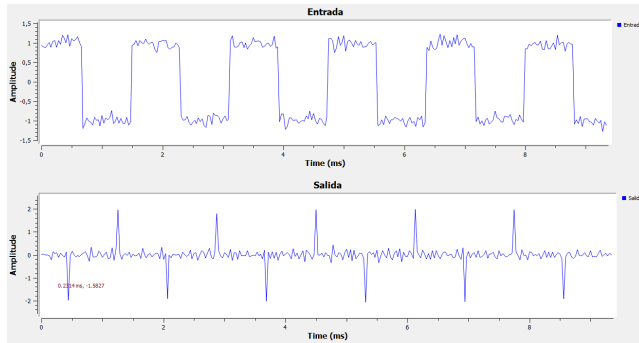


Figure 10. Simulation of differentiator block output with Gaussian noise.

III-C analysis of statistical metrics without adding white Gaussian noise

The flowchart is tested to find the statistical parameters of the digital input function, and the results can be seen in Fig. 11.

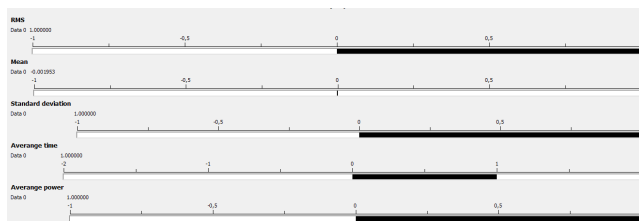


Figure 11. analysis of statistical parameters for the digital input function without adding Gaussian noise.

Values such as mean 0 or standard deviation 1 are retrieved from the model output, which, according to theory (see eq. 3 6), are characteristic of a digital system such as the one initially defined to carry out the experiments.

III-D analysis of statistical metrics with adding white Gaussian noise

Low-amplitude Gaussian white noise introduces only minimal disturbance to the digital signal, so its influence on global statistical metrics such as the mean, RMS value, or standard deviation is practically negligible when working with a high signal-to-noise ratio (SNR) such as that used in this experiment.

III-E Analysis of the influence of sampling frequency on the statistical parameters of digital signals.

This section examines the influence of parameters such as sampling rate in the context of SDR when studying different characteristics of a digital signal. Mathematically everything will depend on the values of the sum of both signals for metrics such as standard deviation, power average or RMS but mean will always be 0.

The sampling rate is fundamental for sampling and representing signals, and it is even more important when working with discrete signals (**pulses**), whose PSD (**sinc**) manifests itself throughout the entire frequency domain, in order to idealize the window. By restricting the PSD to a finite domain of values, the signal begins to undergo distortions, resulting in slower changes. As the number of samples decreases, it becomes necessary to increase the bandwidth due to the proportional increase in signal changes. This is due to the point-to-point multiplication of the PDS of the pulses.

IV CONCLUTIONS

The laboratory provided valuable experience in programming custom blocks within GNURadio, reinforcing the integration of theoretical concepts with practical implementation. By developing accumulator and differentiator blocks, it was possible to better understand the behavior of discrete-time signals and their importance in digital signal processing. The use of Python also facilitated the construction of statistical analysis tools, which supported the evaluation of noise and signal properties in a more complete way.

In addition, the laboratory emphasized the importance of incorporating statistical analysis in the study of discrete signals. By calculating parameters such as mean, RMS, power, and standard deviation, it was possible to obtain a clearer understanding of the influence of noise and the variability present in practical systems. This statistical perspective proved fundamental for controlling and mitigating unwanted effects in signals, highlighting its role as a key tool in noise treatment and digital signal processing applications.

References

- [1] Wikipedia contributors, "Gnu radio — embedded python block," https://wiki.gnuradio.org/index.php/Embedded_Python_Block, 2025, accessed: 2025-09-01.
- [2] Escuela de Ingenierías Eléctrica, Electrónica y de Telecomunicaciones, *Libro de Comunicaciones Plus Volumen I*. Bucaramanga, Colombia: Universidad Industrial de Santander, 2020, material de apoyo académico.